

MediSimplify — MVP Plan

AI-powered health literacy translator | CTRL-V Hackathon

1. Overview

One-liner: MediSimplify turns confusing prescriptions, lab reports, and discharge notes into plain-language summaries you can read or listen to — no account, no typing, no prompt skills required.

The problem: Medical documents are written for clinicians, not patients. Elderly people, non-native speakers, low-literacy users, and anyone under stress (which is everyone in a hospital) routinely misunderstand dosages, warning signs, and follow-up instructions. This causes missed doses, medication errors, and preventable ER visits.

The insight: The technology to fix this (OCR + LLMs) already exists and is even free via ChatGPT — but the people who need it most don't know how to use it that way. MediSimplify removes the prompt. One button: upload → listen.

2. Objectives

1. **Prove the core loop works end-to-end** (photo in → plain-language + audio out) in a live demo.
 2. **Demonstrate a real AI pipeline**, not a thin wrapper — OCR, structured LLM extraction, and TTS chained together.
 3. **Show a built-in safety guardrail** (flagging when a human — pharmacist/doctor — should be consulted) that a generic chatbot wouldn't reliably surface.
 4. **Design for the actual target user:** zero login, minimal text input, large tap targets, audio-first.
 5. **Deliver a demo-able story in under 3 minutes**, mapped directly to judging criteria.
-

3. Target Users

User	Barrier
Elderly patient	Small print, medical jargon, unfamiliar with apps
Non-native speaker	Language barrier on top of medical jargon
Low-literacy user	Can't easily read dense text; needs audio
Anyone post-diagnosis, stressed	Cognitive overload, can't process complex instructions in the moment

4. MVP Scope (48-Hour Build)

Core Features (must-have)

1. **Upload / Capture**
 - Upload a photo or PDF of a prescription, lab report, or discharge summary
 - (Stretch) Take photo directly via camera on mobile web
2. **Text Extraction (OCR)**
 - Run image/PDF through OCR (Tesseract or a vision-capable LLM call)
 - Not building custom OCR — using an existing library/API is fine, since the value-add is downstream
3. **Plain-Language Conversion (LLM)**
 - Structured prompt converts extracted text into a **fixed output schema** every time:
 - **Summary** — what this document is about, in one paragraph
 - **What to do** — checklist of actions (take X pill, at Y time, with/without food)
 - **Warning signs** — when to seek help, in plain terms
 - **Confidence flag** — if OCR text is garbled or content is ambiguous, explicitly say "please confirm with a pharmacist/doctor" instead of guessing
4. **Audio Narration (TTS)**
 - Convert the plain-language summary into speech
 - One "Play" button — no scrolling required to get the info
5. **Minimal UI**
 - One screen: Upload → Loading → Result (text + audio)
 - Large fonts, high contrast, no login wall

Stretch Features (only if time allows)

- Language selector (translate output into Spanish/Hindi/etc.)
- Downloadable/shareable summary (for a caregiver)
- "Explain this word" tap-to-define for individual medical terms

Explicitly Out of Scope (say this out loud to judges — shows maturity)

- Real medical diagnosis or treatment advice
 - Storing/managing patient health records (no persistence needed for MVP)
 - Drug interaction database integration (mention as roadmap item, don't build it)
 - Native mobile app — a responsive mobile web app is enough
-

5. User Flow



6. Suggested Tech Stack (fast to build, demo-friendly)

Layer	Tool
Frontend	React (or plain HTML/CSS/JS) — mobile-first, one page
OCR	Tesseract.js (client-side, no backend needed) or a vision LLM call
LLM	Claude/GPT API with a fixed JSON-schema prompt
TTS	Browser Web Speech API (free, no setup) or ElevenLabs if time allows
Hosting	Vercel/Netlify for instant deploy
Backend	Minimal — a single serverless function to call the LLM API (keep API key server-side)

Keep infra as thin as possible — judges are scoring the *pipeline logic and impact*, not your DevOps.

7. The Guardrail Prompt (core differentiator)

This is what separates you from "just ask ChatGPT." Build a fixed system prompt like:

You are a medical-document simplifier for patients. Always output exactly these four sections: Summary, What To Do, Warning Signs, Confidence Flag. Never invent dosage information not present in the source text. If the extracted text is unclear, incomplete, or you are not fully confident, set Confidence Flag to "Please confirm with a pharmacist or doctor" instead of guessing.

Demo this explicitly: run one clean image (confident output) and one blurry/ambiguous image (triggers the flag). That contrast is your strongest technical + impact talking point.

8. Mapping to Judging Criteria

Criteria	Weight	How MVP addresses it
Impact	30%	Directly reduces medication errors/confusion — a universal, high-stakes access gap
Innovation	25%	Combines OCR + structured LLM + audio + safety-flagging in one zero-friction flow — not just "chat with your PDF"
Technical Implementation	25%	Real multi-step AI pipeline (OCR → structured extraction → TTS), with an explicit confidence/safety mechanism
User Experience	10%	One-screen flow, no login, audio-first, large UI — designed for the actual target user, not a developer
Presentation	10%	Live before/after demo (jargon photo → plain audio) + the blurry-image guardrail moment

9. Demo Script (3 minutes)

1. **(20s)** Problem: show a real, messy prescription photo. "Can you tell me exactly when to take this and what to watch for?"
2. **(60s)** Live demo: upload it → plain-language summary appears → hit play, audio reads it aloud.
3. **(30s)** Show the guardrail: upload a blurry/ambiguous doc → app flags "confirm with pharmacist" instead of guessing.
4. **(30s)** The "why not just ChatGPT" answer: one sentence — *"This is for people who don't know to ask ChatGPT to do this — we removed the prompt."*
5. **(20s)** Close on impact: who this helps and why it matters.

10. Build Order (Hour-by-Hour Suggestion)

Time	Task
Hr 0-2	Set up repo, basic upload UI, wire OCR library
Hr 2-5	Build & test the structured LLM prompt (this is your core IP — spend real time here)
Hr 5-7	Wire TTS playback, connect result screen
Hr 7-9	Polish UI: fonts, contrast, mobile responsiveness
Hr 9-11	Add confidence-flag guardrail + test with a deliberately bad image
Hr 11-13	(Stretch) Language translation toggle
Hr 13-20	Testing, bug fixes, prep sample documents for demo
Hr 20-24	Rehearse demo script, prepare slides/pitch

Next steps once this is locked: I can draft the actual LLM system prompt in full, write the pitch deck outline, or scaffold the starter code.